Информационные системы управления данными и документами

ФИО ст.преподавателя: Тесла-Блозва Святослав Александрович

e-mail: teslablozva@mirea.ru

О чем сегодня пойдет речь?

1. ORM/JPA/Hibernate
2. Spring и Spring Boot
3. Spring Data JPA, Repositories
4. PostgreSQL и Service Layer (@Transactional)
5. Vaadin Grid UI

Проблема Impedance Mismatch (Несоответствие моделей)

Несоответствие между объектно-ориентированной моделью Java (объекты, наследование, ссылки) и реляционной моделью БД (таблицы, строки, внешние ключи).

Сложность ручного маппинга данных при заполнении объектов.`

Что такое ORM

ORM – это техника программирования, которая служит для того, чтобы обеспечивать преобразование данных при их обмене между реляционной базой данных и (в нашем случае) Java.



Что такое JPA?

JPA (Java Persistence API) — программный интерфейс API, входящий (начиная с версии Java 5) в состав платформ Java SE и Java EE. Существует несколько реализаций интерфейса JPA, среди которых наиболее популярным является Hibernate.

История JPA

JPA, или Java Persistence API, представляет собой набор интерфейсов и классов в языке программирования Java, который обеспечивает стандартизированный способ доступа, управления и сохранения данных между объектами Java и реляционной базой данных.

JPA 2.0 начал разрабатываться в 2007 году. Он был обозначен как 2.0, потому что не удалось достичь единого согласия для версии 1.0. Основное внимание уделялось добавлению функциональности, доступной в качестве части известных поставщиков ORM.

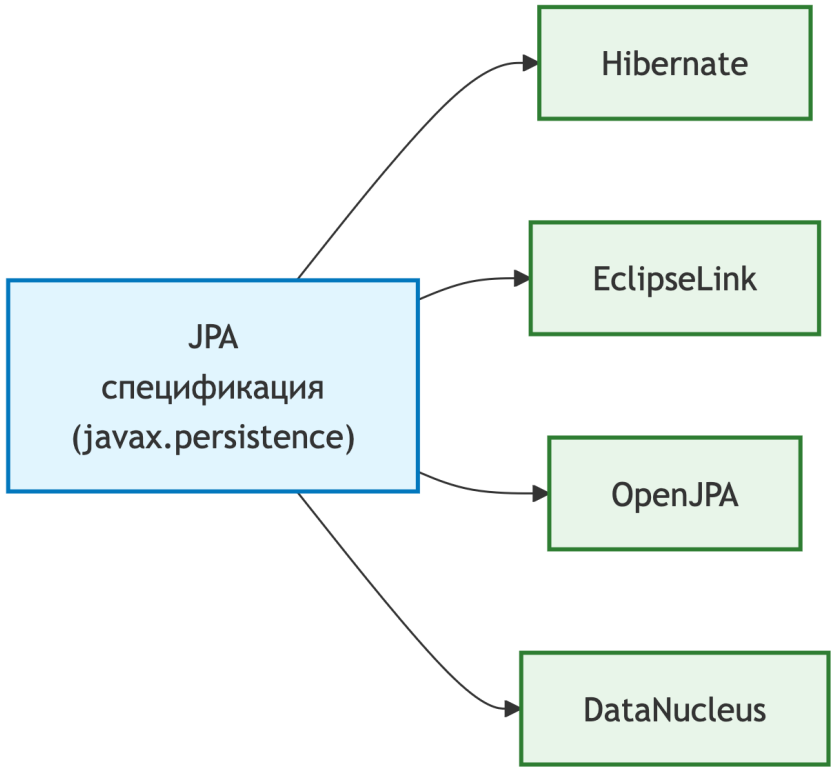
JPA 2.1 был выпущен в июле 2011 года. Некоторые из его ключевых функций включают граф сущностей, хранимые процедуры, преобразователи и др.

JPA 2.2, последнее дополнение к серии JPA, был выпущен в 2017 году. Он включает в себя поддержку новых возможностей, таких как дата и время в Java 8, а также возможность потоковой передачи результатов запросов.

Разработка **JPA** все еще активно продолжается, и мы можем ожидать выпуска новых версий в ближайшем будущем, чтобы улучшить и дополнить эту технологию.

Сейчас актуальна уже **Jakarta Persistence 3.x** — после передачи Java EE в Eclipse Foundation пакеты переименовали из javax.persistence в jakarta.persistence. Это ломающее изменение: при переходе со Spring Boot 2 на Spring Boot 3 нужно менять импорты. Об этом стоит помнить.

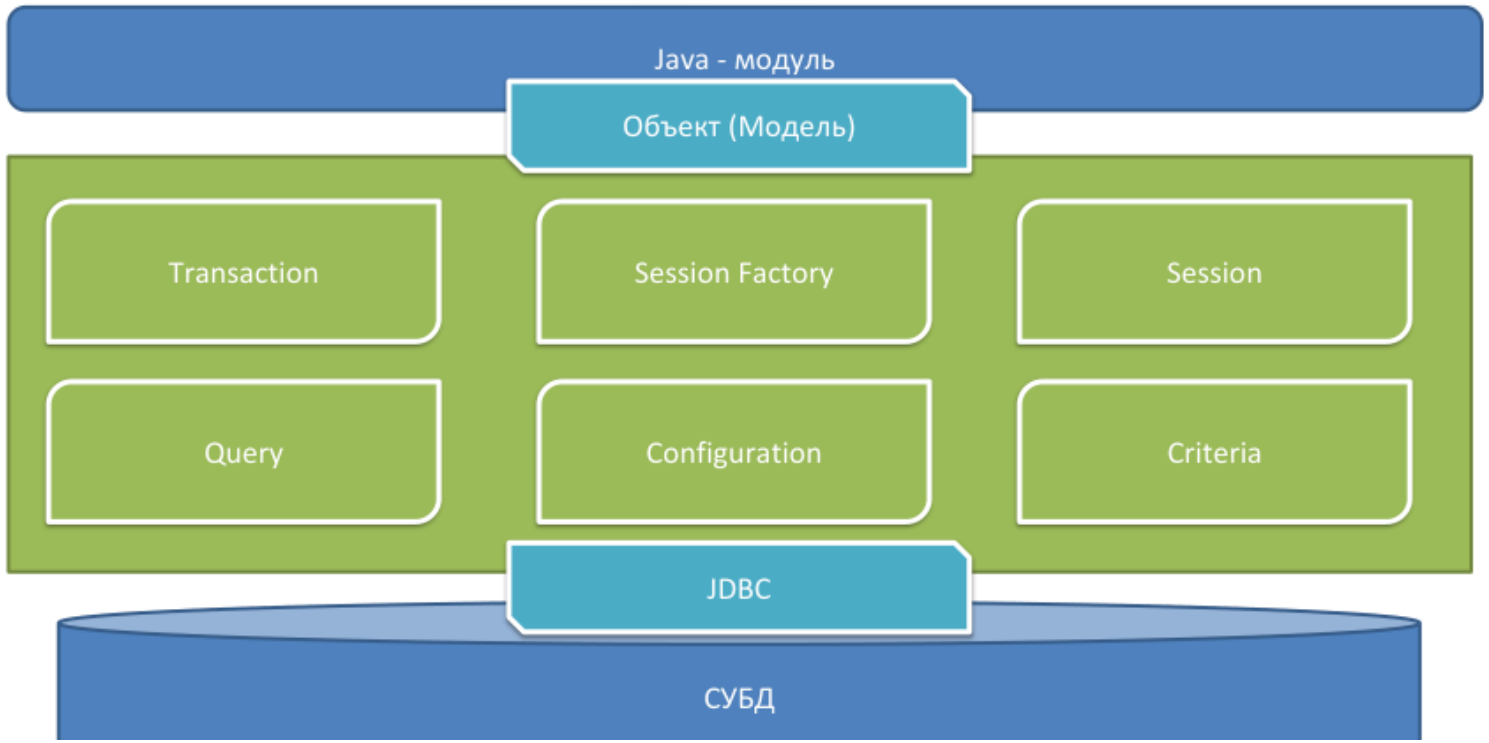
JPA и Hibernate



ORM vs JDBC

- Позволяет нашим бизнес-методам обращаться не к БД, а к Java-классам
- Ускоряет разработку приложения
- Основан на JDBC
- Отделяет SQL-запросы от ОО модели
- Позволяет не думать о реализации БД
- Сущности основаны на бизнес-задачах, а не на структуре БД
- Управление транзакциями

Архитектура JPA и Hibernate (EntityManagerFactory, EntityManager, Persistence Context)



Что такое JPQL

Java Persistence Query Language (JPQL) — платформенно-независимый объектно- ориентированный язык запросов, являющийся частью спецификации Java Persistence API (JPA). JPQL используется для написания запросов к сущностям, хранящимся в реляционной базе данных.

Разница:

SQL:

SELECT * FROM documents WHERE title LIKE '%отчёт%'

JPQL:

SELECT d FROM Document d WHERE d.title LIKE '%отчёт%'

Что такое JPQL

Этот язык похож на SQL, но использует объектную модель Java, а не SQL таблицы.

JPQL основан на Hibernate Query Language (HQL), более раннем не стандартизованном языке запросов, включённом в библиотеку объектно-реляционного отображения Hibernate. Hibernate и HQL были созданы до появления спецификации JPA. JPQL является подмножеством языка запросов HQL.

Понятие JPA Entity и требования к POJO. Hibernate. Persistent classes

Для того, чтобы сделать работу с Hibernate максимально удобной и эффективной, следует использовать программную модель Простых Старых Java Объектов (Plain Old Java Object – POJO). Существуют определённые требования к POJO классам.

Понятие JPA Entity и требования к POJO. Hibernate. Persistent classes

Вот самые главные из них:

- Все классы должны иметь ID для простой идентификации наших объектов в БД и в Hibernate. Это поле класса соединяется с первичным ключом (primary key) таблицы БД.
- Все POJO – классы должны иметь конструктор по умолчанию (пустой).
- Все поля POJO – классов должны иметь модификатор доступа private иметь набор getter- и setter-методов в стиле JavaBean.
- POJO – классы не должны содержать бизнес-логику.

Базовые аннотации уровня класса

Аннотация @Entity используется для обозначения класса как сущности.

- @Table используется для указания имени таблицы в базе данных.
- @Id указывает на атрибут, который является идентификатором.
- @GeneratedValue используется для автоматической генерации значений идентификатора. GenerationType определяет метод генерации значения.
- @Column используется для сопоставления атрибутов класса с столбцами таблицы.

Базовые аннотации уровня класса. Определение первичного ключа (@Id)

```
@Id
private Long id;


// или UUID
@Id
@GeneratedValue
private UUID id;
```

Базовые аннотации уровня класса. Стратегии генерации ключей (@GeneratedValue)

- AUTO — провайдер сам решает.
- IDENTITY — автоинкремент СУБД (SERIAL в PostgreSQL, AUTO_INCREMENT в MySQL).
- SEQUENCE — последовательность (CREATE SEQUENCE).
- TABLE — отдельная таблица для счётчиков (медленно, для совместимости).

Базовые аннотации уровня класса. Стратегии генерации ключей (@GeneratedValue)

Пример

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "doc_seq")
@SequenceGenerator(name = "doc_seq", sequenceName = "document_seq", allocationSize = 50)
private Long id;
```

Сравнение IDENTITY и SEQUENCE

- GenerationType.IDENTITY — использует автоинкрементную колонку БД (AUTO_INCREMENT, SERIAL).
- GenerationType.SEQUENCE — использует последовательность БД (SEQUENCE).
- IDENTITY: ID генерируется СУБД **во время INSERT**.
- SEQUENCE: Hibernate запрашивает ID **заранее** из последовательности.
- Из-за этого IDENTITY **отключает batch-inserts**: Hibernate не может сгруппировать вставки, так как ID нужен до завершения INSERT.
- SEQUENCE **поддерживает batch-inserts** и лучше подходит для PostgreSQL.
- AUTO — выбор стратегии провайдером.
- TABLE — отдельная таблица для счетчиков, медленно.

Сравнение IDENTITY и SEQUENCE

```
// IDENTITY
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// SEQUENCE
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "doc_seq")
@SequenceGenerator(name = "doc_seq", sequenceName = "document_seq",
allocationSize = 50)
private Long id;
```

Базовые аннотации уровня класса. Маппинг полей сущности (@Column)

@Column сопоставляет поле класса со столбцом таблицы.

Основные параметры:

- name — имя колонки;
- nullable — разрешено ли NULL;
- length — длина строки;
- unique — уникальность значения;
- insertable / updatable — участие в INSERT/UPDATE;
- columnDefinition — произвольный SQL-тип;
- precision / scale — для BigDecimal.

Если имя поля совпадает с именем колонки, @Column можно не указывать.

Базовые аннотации уровня класса. Маппинг полей сущности (@Column)

```
@Column(name = "title", nullable = false, length = 255, unique = true)

private String title;



@Column(name = "page_count", nullable = false)

private Integer pageCount;
```

Работа с типами данных: Enum и Date/Time

@Enumerated(EnumType.STRING) — хранит **имя** константы. Предпочтительно.

@Enumerated(EnumType.ORDINAL) — хранит **индекс**. Опасно при изменении порядка констант.

Java 8 Time API (LocalDate, LocalDateTime, Instant) поддерживается JPA 2.2+ **без** @Temporal.

Для старых версий JPA: @Temporal(TemporalType.DATE) и т.п.

ZonedDateTime часто требует @Convert.

Работа с типами данных: Enum и Date/Time

```
public enum DocumentStatus {
    DRAFT, PUBLISHED, ARCHIVED
}

@Enumerated(EnumType.STRING)
@Column(nullable = false, length = 20)
private DocumentStatus status;

@Column(name = "created_date", nullable = false)
private LocalDateTime createdDate;

@Column(name = "publish_date")
private LocalDate publishDate;
```

Исключение полей из маппинга (@Transient)

@Transient — поле **не сохраняется** в БД и не участвует в маппинге.

Используется для:

- вычисляемых полей;
- временных данных;
- UI-состояния (например, выбранная строка);
- кэша.

Не путать с Java-ключевым словом transient.

JPA-аннотация @Transient имеет приоритет.

Исключение полей из маппинга (@Transient)

```
@Transient
private String formattedCreatedDate;

@Transient
private boolean selected;
```

Обзор состояний объектов в JPA/Hibernate

Состояния объектов сессий:

- **transient**

Это новый экземпляр класса, который не привязан к сессии и ещё не представлен в БД. Он не имеет значения, по которому может быть идентифицирован.

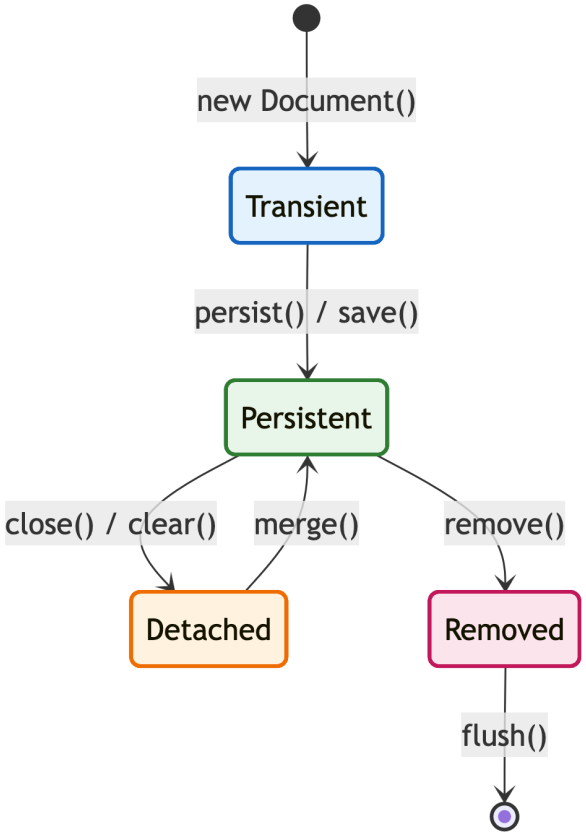
- **persistent**

Устойчивый экземпляр класса представлен в БД, а значение идентификатора связано с сессией.

- **detached**

После того, как сессия закрыта, экземпляр класса становится отдельным, независимым экземпляром класса.

Состояния объекта (переходы)



Состояние Transient

- new Document(), ID = null.
- Не привязан к EntityManager.
- Изменения нигде не сохраняются.

Как получить Transient-объект? Просто создать через new:

```
Document doc = new Document();
doc.setTitle("Отчёт");
doc.setStatus(DocumentStatus.DRAFT);
```

Состояние Persistent / Managed

- Объект в Persistence Context.
- **Dirty Checking**: изменение поля автоматически приведёт к UPDATE при flush().
- entityManager.persist(doc) переводит в это состояние.

```
@Transactional
public void renameDocument(Long id) {
    Document doc = repository.findById(id).orElseThrow();
    doc.setTitle("Новое название");
    // НЕ вызываем save()!
    // при выходе из метода транзакция закоммитится,
    // Hibernate увидит изменение и сделает UPDATE
}
```

Состояние Detached и merge()

- Сессия закрыта / clear() / detach().
- Есть ID, есть строка в БД, но изменения не отслеживаются.
- entityManager.merge(doc) возвращает **новый** управляемый объект; исходный остаётся detached.

```java
@GetMapping("/documents/{id}")
public Document getDocument(@PathVariable Long id) {
    return repository.findById(id).orElseThrow();
}
// после выхода из метода объект Detached
```

```java
@PutMapping("/documents/{id}")
public Document updateDocument(@RequestBody Document doc) {
    return repository.save(doc); // save = merge, если ID != null
}
```

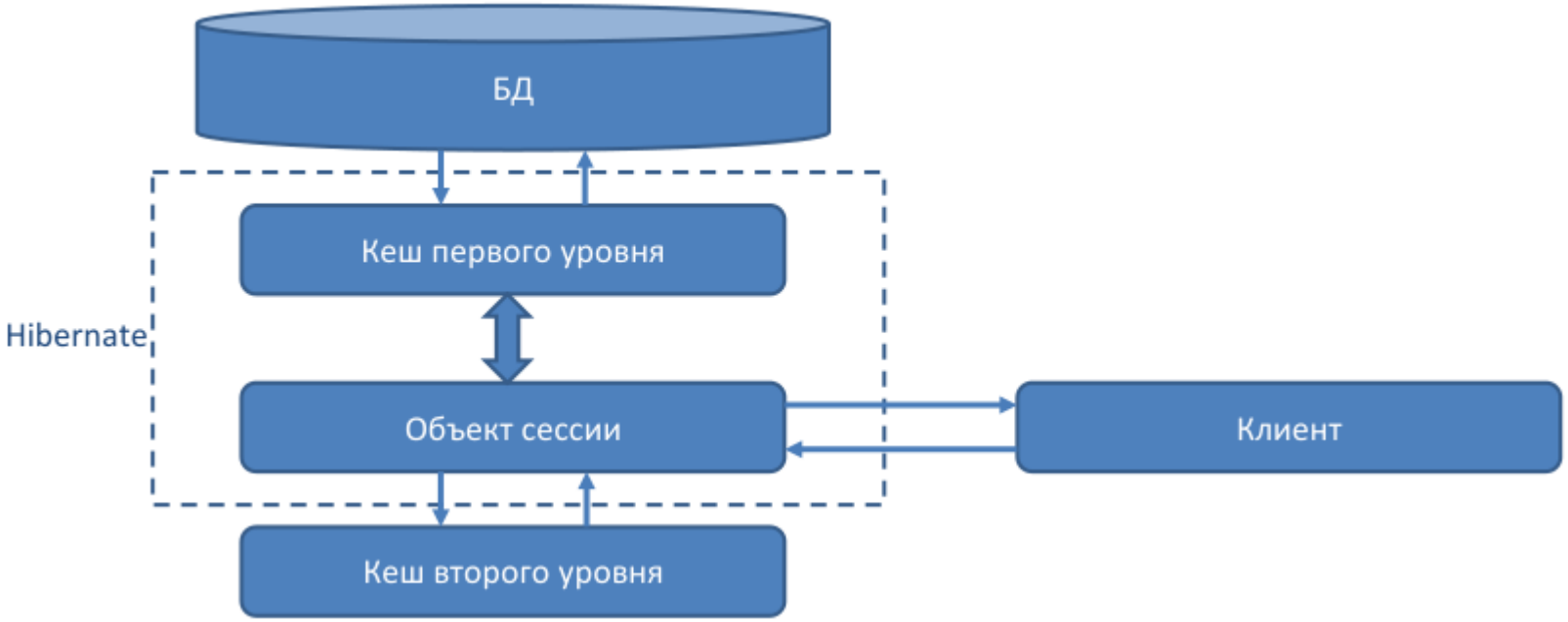
Состояние Removed

- entityManager.remove(doc) — планирует DELETE.
- До flush() объект ещё в БД.
- После flush() — удалён.

```
@Transactional
public void deleteDocument(Long id) {
    Document doc = repository.findById(id).orElseThrow();
    repository.delete(doc);
}
```

Кэширование в JPA

Кеширование является одним из способов оптимизации работы приложения, ключевой задачей которого является уменьшить количество прямых обращений к базе данных.



Почему PostgreSQL?

- Полное соответствие стандарту SQL.
- Продвинутые типы: JSONB, массивы, hstore, диапазоны.
- Полнотекстовый поиск (tsvector, tsquery).
- Расширения: PostGIS, pg_trgm, uuid-ossp.
- Надёжность, MVCC, репликация.
- Открытая лицензия, хорошая производительность.
- Отличная поддержка в Spring Boot / Hibernate.

Spring Framework

Spring Framework — каркас для Java-приложений, появился в 2003 году (Rod Johnson).

Главная идея — **Inversion of Control (IoC)**: не код создаёт объекты, а контейнер Spring.

Dependency Injection (DI) — контейнер сам передаёт зависимости в конструктор или поле.

Бин — объект, управляемый контейнером Spring.

Стереотипы для регистрации бинов:

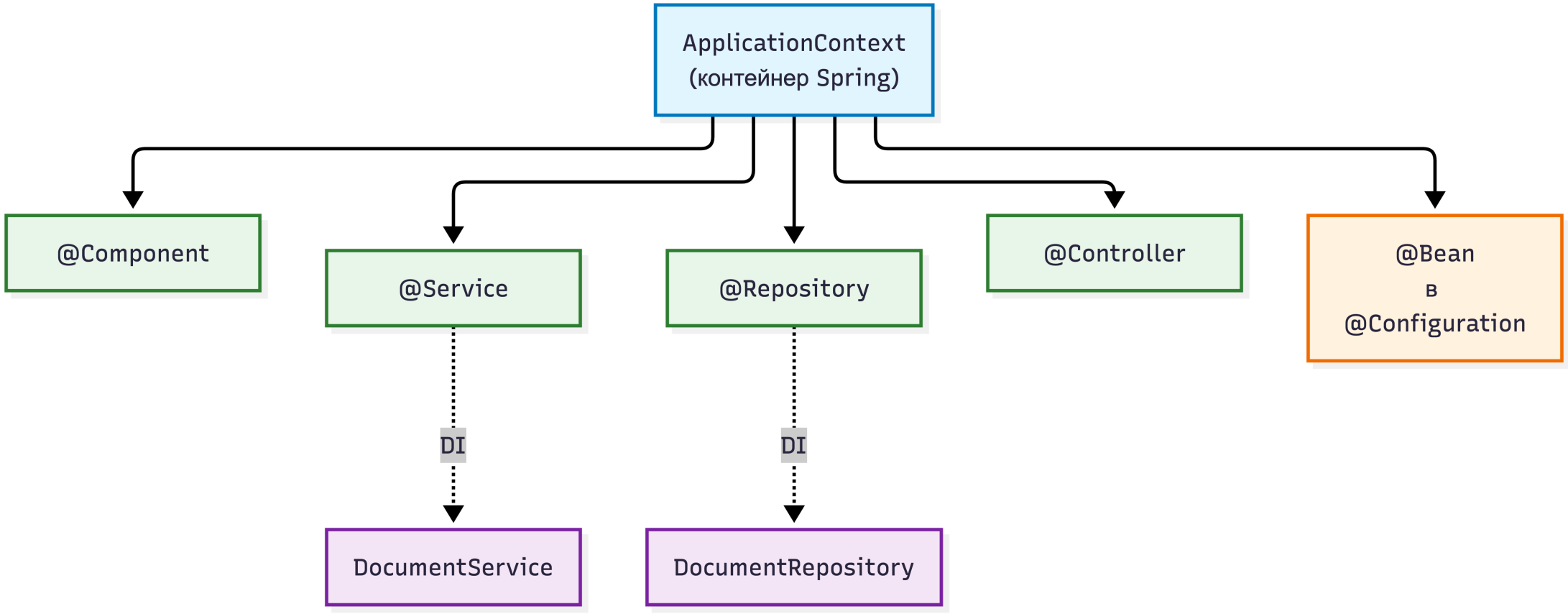
- @Component — общий бин;
- @Service — бизнес-логика;
- @Repository — доступ к данным;
- @Controller / @RestController — веб-слой.

@Configuration + @Bean — ручная регистрация бинов в Java-конфигурации.

AOP — сквозная функциональность (транзакции, логирование, security) выносится в аспекты — на этом основан @Transactional.

ApplicationContext — сам контейнер, хранилище бинов.

Spring Framework



Spring Framework. Пример DI

```
@Service
public class DocumentService {

    private final DocumentRepository repository;

    // Spring сам передаст DocumentRepository
    public DocumentService(DocumentRepository repository) {
        this.repository = repository;
    }
}
```

Spring = IoC-контейнер + DI + AOP. Spring Boot = Spring + автоконфигурация + стартеры.

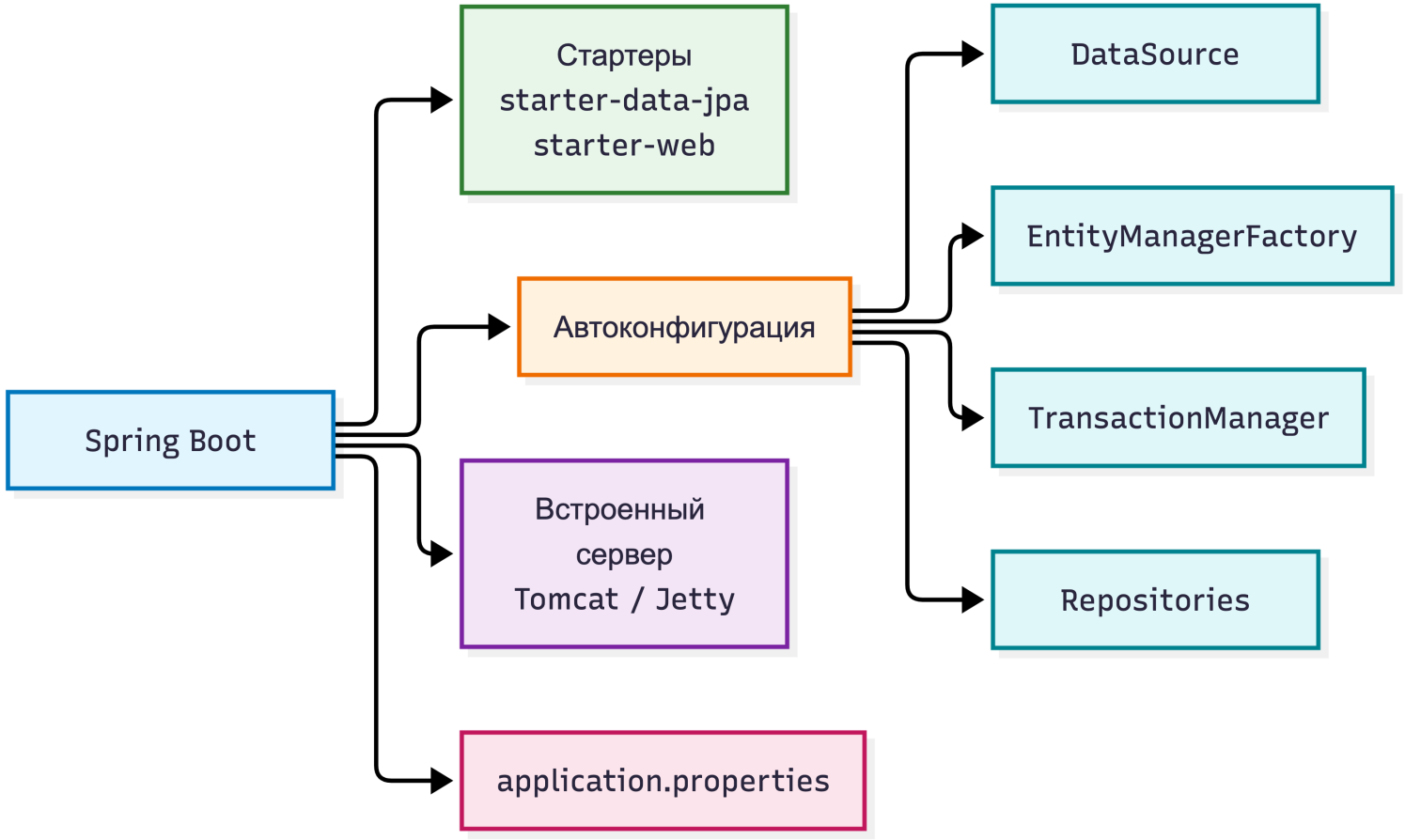
Spring Boot: что это и зачем

- **Spring Boot** — надстройка над Spring Framework, ускоряющая создание приложений.

Убирает ручную настройку XML и Java-конфигурации.

- **Стартеры** (spring-boot-starter-*) — готовые наборы зависимостей «под задачу».
- **Автоконфигурация** — Spring Boot сам создаёт нужные бины (DataSource, EntityManagerFactory, TransactionManager, репозитории).
- **Встроенный сервер** (Tomcat / Jetty / Undertow) — приложение запускается как обычный main().
- **application.properties / application.yml** — единое место для всех настроек.
- **Actuator** — готовые endpoint'ы для мониторинга и health-check

Spring Boot: что это и зачем



Spring Boot. Стартеры и точка входа

- **spring-boot-starter-parent** — родительский POM, управляет версиями всех зависимостей.
- **spring-boot-starter-data-jpa** — тянет JPA, Hibernate, HikariCP, spring-data-jpa.
- **spring-boot-starter-web** — Spring MVC + встроенный Tomcat.
- **vaadin-spring-boot-starter** — интеграция Vaadin со Spring Boot.
- Драйвер БД (org.postgresql:postgresql) подключается отдельно.
- Точка входа — класс с **@SpringBootApplication** и вызовом SpringApplication.run(...).

Spring Boot. Стартеры и точка входа

```xml
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
</parent>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa
            </artifactId>
    </dependency>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
    </dependency>
    <dependency>
        <groupId>com.vaadin</groupId>
        <artifactId>vaadin-spring-boot-starter
            </artifactId>
    </dependency>
</dependencies>
```

Spring Boot. Стартеры и точка входа

```
@SpringBootApplication
public class DocumentsApplication {

    public static void main(String[] args) {
        SpringApplication.run(DocumentsApplication.class, args);
    }
}
```

- @SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan.
- SpringApplication.run(...) запускает контейнер Spring и встроенный сервер.

Spring vs Spring Boot

Spring Framework	Spring Boot
Нужно вручную настраивать DataSource, EntityManagerFactory, TransactionManager	Настраивает автоматически
Конфигурация через XML или Java-классы с @Configuration	application.properties + автоконфигурация
Нужен внешний сервер (Tomcat)	Встроенный сервер
Управление версиями зависимостей вручную	Через spring-boot-starter-parent
Гибче, но больше шаблонного кода	Быстрее старт, меньше кода

Подключение драйвера PostgreSQL

Maven

```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.3</version>
</dependency>
```

Gradle

```
implementation
'org.postgresql:postgresql:42.7.3'
```

Конфигурация подключения в application.properties

- spring.datasource.url=jdbc:postgresql://localhost:5432/docdb
- spring.datasource.username=postgres
- spring.datasource.password=secret
- spring.datasource.driver-class-name=org.postgresql.Driver

Настройка JPA/Hibernate в Spring Boot

- spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
- spring.jpa.show-sql=true
- spring.jpa.hibernate.ddl-auto=update
- spring.jpa.properties.hibernate.format_sql=true

Режимы spring.jpa.hibernate.ddl-auto

Режим	Что делает	Когда использовать
none	Ничего	В продакшене
validate	Проверяет схему	В продакшене
update	Обновляет схему	В разработке
create	Пересоздает при старте	Тесты
create-drop	Пересоздаёт и удаляет при остановке	Тесты

spring.jpa.hibernate.ddl-auto=update

Модуль Spring Data JPA

Spring Data JPA — часть Spring Data, упрощает доступ к БД.

Основная идея: **репозиторий** — интерфейс, реализация создаётся автоматически.

Компоненты:

- **Spring Data Commons** — общая инфраструктура;
- **Spring Data JPA** — интерфейсы репозиториев для JPA;
- **JPA Provider** (Hibernate) — выполняет запросы к БД.

Убирает boilerplate: EntityManager, транзакции, ручные запросы.

Позволяет объявлять методы вида findBy....

Классический JDBC подход vs Spring Data JPA

```java
public class DocumentRepositoryImpl {
    public Document findById(Long id) {
        String sql = "SELECT * FROM documents WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    Document doc = new Document();
                    doc.setId(rs.getLong("id"));
                    doc.setTitle(rs.getString("title"));
                    // ... ещё 20 полей
                    return doc;
                }
            }
        }
        return null;
    }
    // ... ещё 20 методов
}
```

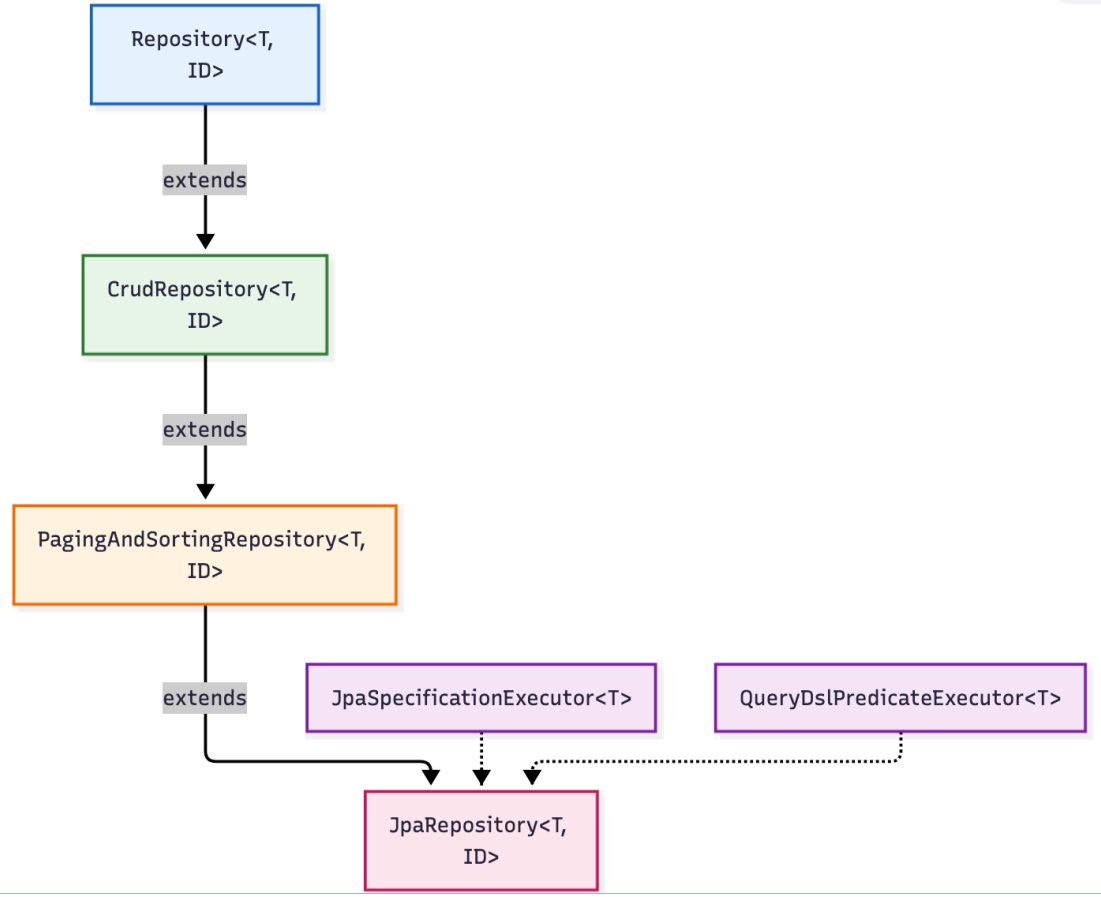
```java
public interface DocumentRepository extends JpaRepository<Document, Long> {
    List<Document> findByTitleContainingIgnoreCase(String title);
}
```

Модуль Spring Data JPA

```
public interface DocumentRepository extends
JpaRepository<Document, Long> {
    List<Document> findByTitleContainingIgnoreCase(String title);
}
```



Иерархия интерфейсов Spring Repository



Пример объявления:

```
public interface DocumentRepository
        extends JpaRepository<Document, Long>,
                JpaSpecificationExecutor<Document> {
}
```

Встроенные CRUD-методы JpaRepository

Метод	Назначение
save(S)	INSERT / UPDATE
saveAll(Iterable<S>)	Пакетное сохранение
findById(ID)	Поиск по ID, Optional<T>
findAll()	Все записи
findAll(Sort) / findAll(Pageable)	С сортировкой / пагинацией
count()	Кол-во записей
existsById(ID)	Проверка наличия
deleteById(ID)	Удаление
delete(T)	Удаление сущности
flush()	Принудительный flush

Автоматические производные запросы

- Spring Data JPA генерирует запрос по имени метода.
- Работает без @Query.
- Начинается с find..., read..., query..., count..., get..., delete..., remove....

Пример:

```
public interface DocumentRepository extends JpaRepository<Document, Long> {
    List<Document> findByTitleContainingIgnoreCase(String title);
    List<Document> findByCreatedDateBetween(LocalDateTime from,
LocalDateTime to);
}
```

Ключевые слова в Derived Queries

Ключевое слово	Пример
And / Or	findByTitleAndStatus
Containing	findByTitleContaining
IgnoreCase	findByTitleIgnoreCase
Between	findByCreatedDateBetween
StartingWith / EndingWith	findByTitleStartingWith
OrderBy...Asc/Desc	findByStatusOrderByCreatedDateDesc

Кастомные запросы с аннотацией @Query

```
public interface UserRepository extends JpaRepository<User, Long> {

  @Query("select u from User u where u.emailAddress = ?1")
  User findByEmailAddress(String emailAddress);

  @Query("select u from User u where u.firstname like %?1")
  List<User> findByFirstnameEndsWith(String firstname);

  @Query("select u from User u where u.firstname = :firstname or u.lastname = :lastname")
  User findByLastnameOrFirstname(@Param("lastname") String lastname,
                                 @Param("firstname") String firstname);
}
```

Преимущества JpaRepository над ручным JDBC

- Сокращение кода до **80%**.
- Нет Connection, PreparedStatement, ResultSet.
- Автоматическое управление транзакциями.
- Встроенная пагинация и сортировка.
- Автогенерация запросов по имени метода.
- Единая обработка исключений (DataAccessException).
- Легче тестировать.

Зачем нужен слой сервисов (@Service)?

- Отделяет бизнес-логику от UI и Repository.
- Переиспользование бизнес-правил в разных UI.
- Управление транзакциями (@Transactional).
- Упрощает unit-тестирование.
- UI не должен знать о БД.
- Repository — только доступ к данным, без логики.

Принципы Clean Architecture и SoC

SoC — разделение ответственности между слоями.

Слои:

- UI (View) — отображение;
- Service — бизнес-логика;
- Repository — доступ к данным;
- Database — хранение.

Зависимости направлены к бизнес-логике.

UI не знает о БД, Repository не знает о UI.

Упрощает тестирование и замену технологий.

Управление транзакциями (@Transactional)

- @Transactional на классе — для всех методов.
- @Transactional(readOnly = true) — оптимизация чтения.
- rollbackFor / noRollbackFor — управление откатом.
- По умолчанию откат при RuntimeException.
- Propagation.REQUIRED — по умолчанию.

```
@Transactional
public void publish(Long id) {
    Document doc = repository.findById(id).orElseThrow();
    doc.setStatus(DocumentStatus.PUBLISHED);
    // UPDATE произойдёт автоматически при commit
}
```

Spring Boot Auto-configuration

Spring Boot автоматически создаёт:

- DataSource;
- EntityManagerFactory;
- TransactionManager;
- репозитории Spring Data JPA.

Работает при наличии зависимостей:

- spring-boot-starter-data-jpa;
- драйвер БД (org.postgresql:postgresql).

Настройки берутся из application.properties / application.yml.

Можно переопределить любой бин через @Bean.

Spring Boot Auto-configuration
Пример application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/docdb

spring.datasource.username=postgres

spring.datasource.password=secret

spring.datasource.driver-class-name=org.postgresql.Driver



spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect

spring.jpa.show-sql=true

spring.jpa.hibernate.ddl-auto=update
```

Компонент Grid в Vaadin

- Grid<T> — табличный компонент Vaadin.
- Поддерживает: сортировку, фильтрацию, lazy loading, выбор строк, редактирование.
- Может работать в двух режимах: **in-memory** (setItems(List)) и **lazy** (setItems(DataProvider)).
- Автоматически подстраивается под размер контейнера.
- Колонки настраиваются вручную через addColumn(...).

Инициализация Grid

```
Grid<Document> grid = new Grid<>(Document.class, false);

grid.addColumn(Document::getId).setHeader("ID").setWidth("80px");

grid.addColumn(Document::getTitle).setHeader("Название").setAutoWidth(
true);

grid.addColumn(Document::getCreatedDate).setHeader("Дата создания");

grid.addColumn(Document::getStatus).setHeader("Статус");

grid.setHeight("500px");
```

Ручная настройка колонок (addColumn)

```
Grid.Column<Document> titleCol =
grid.addColumn(Document::getTitle)
        .setHeader("Название")
        .setSortable(true)
        .setAutoWidth(true);
```

Форматирование данных в колонках

```
grid.addColumn(doc -> doc.getCreatedDate()
        .format(DateTimeFormatter.ofPattern("dd.MM.yyyy HH:mm")))
    .setHeader("Дата");


grid.addColumn(doc -> doc.getStatus().name())
    .setHeader("Статус");
```

Передача данных через grid.setItems(...)

grid.setItems(documentService.findAll());

Подходит только для небольших объёмов (до нескольких тысяч строк).

Это **in-memory** режим — подходит только для небольших таблиц.

Для больших данных нужен DataProvider

Концепция Ленивой загрузки (Lazy Loading) в Vaadin Grid

- Grid загружает данные **порциями** по мере прокрутки.
- Каждый запрос — отдельная страница (PageRequest).
- На сервер уходит: offset, limit, sort, filter.
- В память не попадает вся таблица — только видимые строки + буфер.

Важность Lazy Loading для больших объемов

- Позволяет работать с **миллионами** записей.
- Защищает от OutOfMemoryError.
- Снижает нагрузку на сеть.
- Снижает нагрузку на БД (только нужные строки).
- Ускоряет отклик UI.

Использование DataProvider и ConfigurableFilterDataProvider

```
DataProvider<Document, Void> dataProvider = DataProvider.fromCallbacks(
   query -> {
      PageRequest pageRequest = PageRequest.of(
            query.getOffset() / query.getLimit(),
            query.getLimit(),
            Sort.by(Sort.Direction.DESC, "createdDate"));
      return documentService.findAll(pageRequest).stream();
   },
   query -> (int) documentService.count()
);

ConfigurableFilterDataProvider<Document, Void, String> filtered =
      dataProvider.withConfigurableFilter();

grid.setItems(filtered);
```

Поле поиска TextField

- TextField filterText = new TextField();
- filterText.setPlaceholder("Поиск по названию...");
- filterText.setClearButtonVisible(true);
- filterText.setValueChangeMode(ValueChangeMode.LAZY);

Слушатель событий ValueChangeListener

- filterText.addValueChangeListener(e -> updateList());

Режимы ValueChangeMode:

- EAGER — на каждое нажатие.
- LAZY — с задержкой (по умолчанию 300 мс).
- TIMEOUT — по таймауту.

Реализация метода updateList()

```
• private void updateList() {
•     grid.setItems(service.findByTitle(filterText.getValue()));
• }
```

Встроенная сортировка по колонкам

- grid.addColumn(Document::getTitle)
- .setHeader("Название")
- .setSortable(true);

- grid.addColumn(Document::getCreatedDate)
- .setHeader("Дата")
- .setSortable(true);

Настройка сортировки по умолчанию

```
• Sort defaultSort = Sort.by(Sort.Direction.DESC, "createdDate");



• @Transactional(readOnly = true)
• public List<Document> findAll() {
•     return repository.findAll(defaultSort);
• }
```

Интеграция сортировки Vaadin с Pageable / Sort в Spring Data JPA

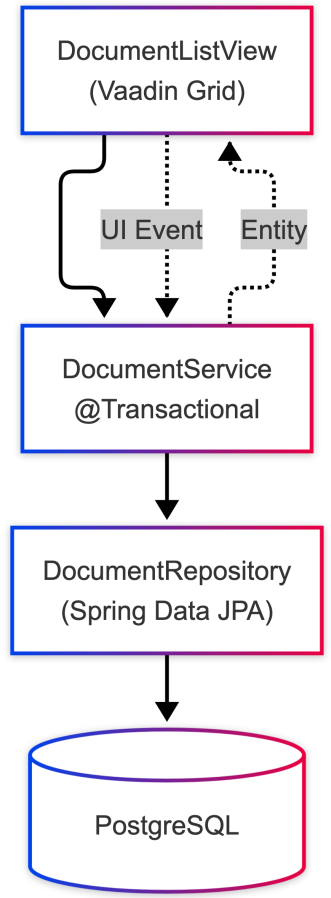
```
Query<Document, Void> query = ...; // из Grid


Sort sort = Sort.unsorted();
for (QuerySortOrder order : query.getSortOrders()) {
    sort = sort.and(Sort.by(
            Sort.Direction.fromString(order.getDirection().name()),
            order.getSorted()));
}


PageRequest pageable = PageRequest.of(
        query.getOffset() / query.getLimit(),
        query.getLimit(),
        sort);


List<Document> page = repository.findAll(pageable).getContent();
```

Полная схема архитектуры приложения



Спасибо за внимание!